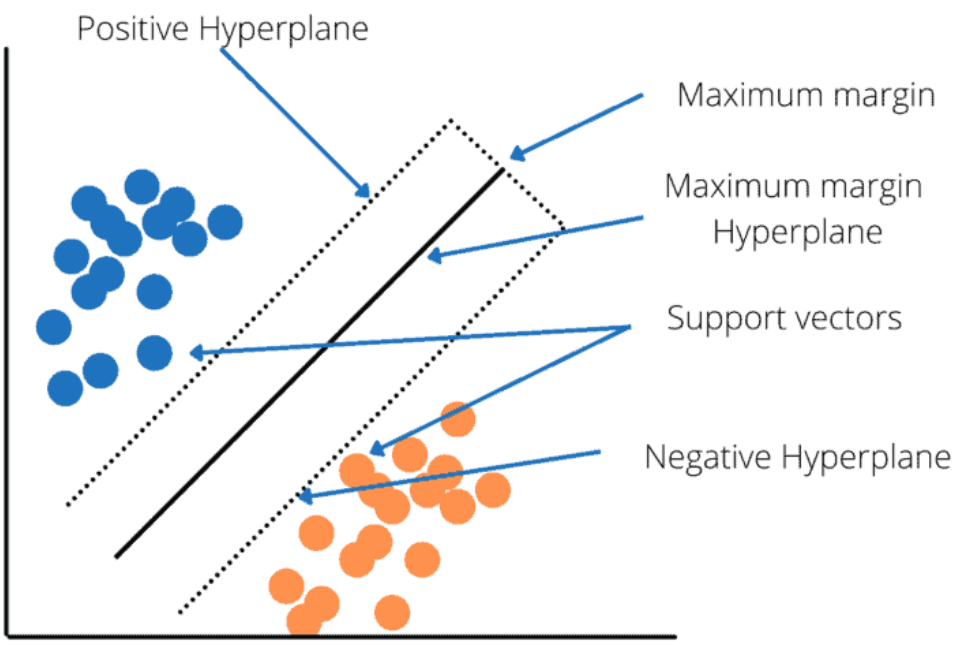
Support Vector Machines (Linear SVM)

1. Linear Support Vector Machines (SVM)

Support Vector Machines (SVM) is a supervised learning algorithm used for both **classification** and **regression** tasks. A **Linear SVM** is a type of SVM where the decision boundary is a straight line (in 2D) or a hyperplane (in higher dimensions).



1.1 Working of Linear SVM

- Linear SVM finds the **optimal hyperplane** that maximizes the margin between different classes.
- The margin is the distance between the hyperplane and the nearest **support vectors** (data points closest to the decision boundary).
- The larger the margin, the better the generalization to unseen data.

1.2 Mathematical Formulation of Linear SVM

A hyperplane in **n-dimensional space** is defined as:

$$w \cdot x + b = 0$$

Where:

- w is the weight vector
- x is the feature vector
- b is the bias term

For classification, SVM ensures:

$$y_i(w \cdot x_i + b) \geq 1$$

Where:

- y_i is the class label (-1 or 1)
- x_i is the input data point

The objective function for optimization is: $\min \frac{1}{2}||w||^2$ subject to the constraint that all points are correctly classified with maximum margin.

2. Scikit-Learn Implementation of Linear SVM

```python
from sklearn.svm import SVC
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Generate dataset
X, y = make_classification(n_samples=100, n_features=2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train a Linear SVM model
model = SVC(kernel='linear')
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

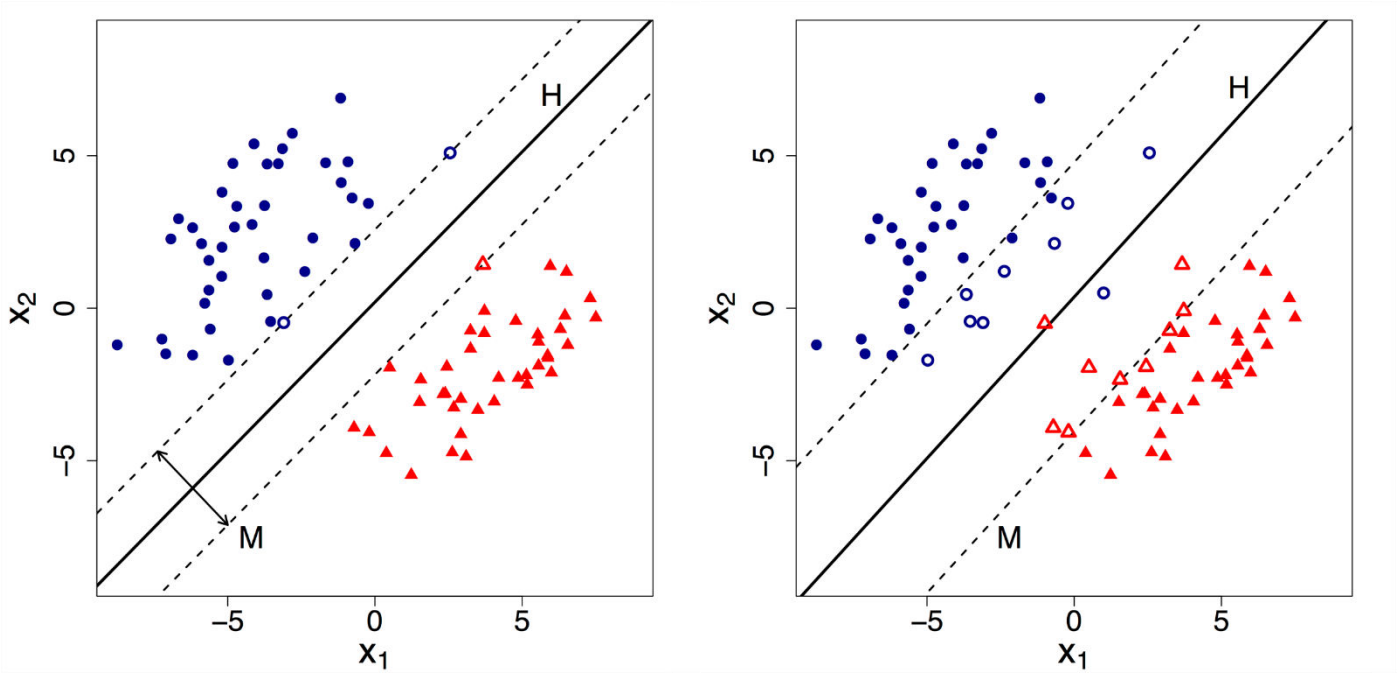
3. Linear Classification

Linear classification refers to classifying data using a **straight-line decision boundary**. Linear SVM is effective when the data is **linearly separable**. If data points can be separated by a **single straight line**, then a linear SVM is suitable.

3.1 Hyperplane and Support Vectors

- **Hyperplane:** A decision boundary that separates classes.
- **Support Vectors:** Data points closest to the hyperplane.

A dataset in **2D space** forms a **straight line**, while in **3D space**, it forms a **plane**.



4. Kernel-Based Classification

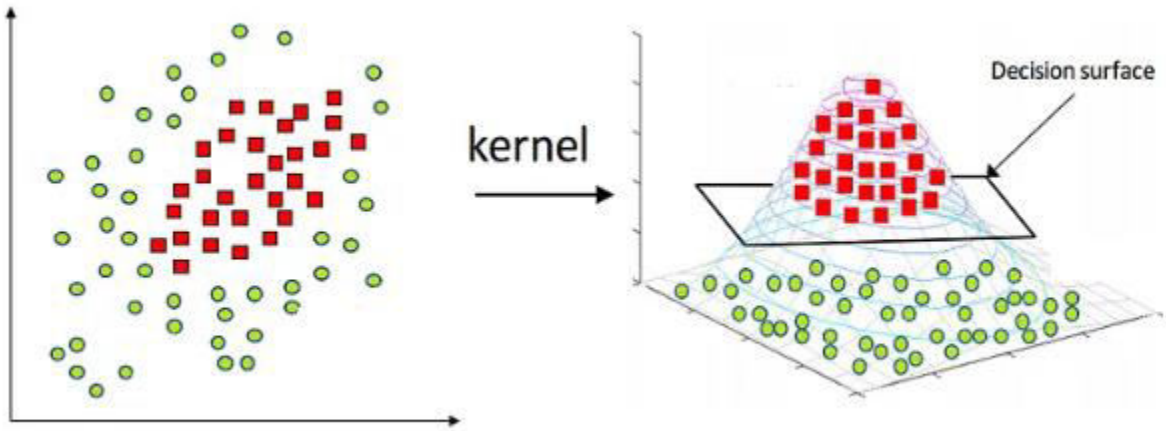
When data is **not linearly separable**, we use **kernel tricks** to transform data into a higher-dimensional space where it becomes linearly separable.

4.1 Types of Kernels

✅**Radial Basis Function (RBF) Kernel**

$$K(x, x') = \exp(-\gamma ||x - x'||^2)$$

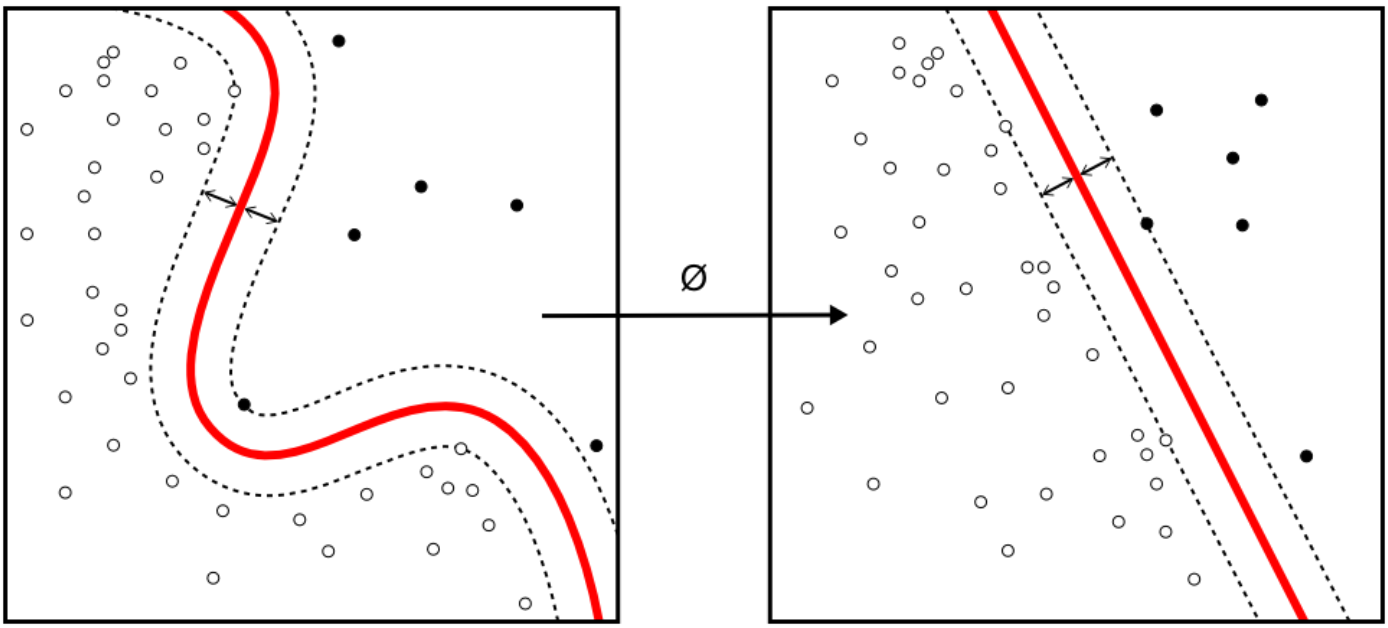
- Adds a **non-linear transformation** to make data linearly separable.
- Works well in most real-world scenarios.



✅Polynomial Kernel

$$K(x, x') = (x \cdot x' + c)^d$$

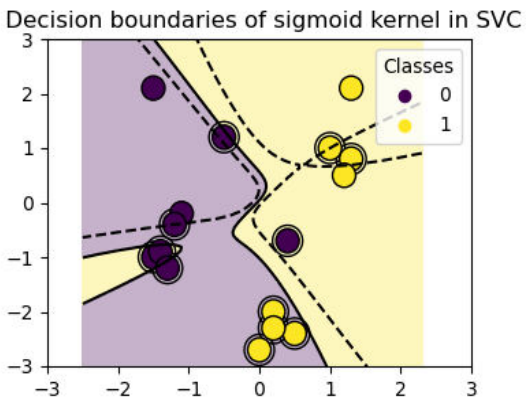
- Used when data exhibits polynomial relationships.
- Degree controls complexity.



✅Sigmoid Kernel

$$K(x, x') = \tanh(\alpha x \cdot x' + c)$$

- Inspired by neural networks, used in some cases of text classification.



✅**Custom Kernels**

- We can define custom kernels based on specific domain knowledge.

4.2 Scikit-Learn Implementation of Kernel SVM

```
# Using RBF Kernel
model_rbf = SVC(kernel='rbf', gamma='scale')
model_rbf.fit(X_train, y_train)
y_pred_rbf = model_rbf.predict(X_test)
print("RBF Kernel Accuracy:", accuracy_score(y_test, y_pred_rbf))
```

5. Non-Linear SVM Examples

Example: XOR Problem (Non-Linearly Separable Data)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC

# XOR dataset
X = np.array([[0,0], [0,1], [1,0], [1,1]])
y = np.array([0, 1, 1, 0])

# Train with RBF kernel
clf = SVC(kernel='rbf', gamma='auto')
clf.fit(X, y)

# Plot decision boundary
xx, yy = np.meshgrid(np.linspace(-1, 2, 100), np.linspace(-1, 2, 100))
Z = clf.decision_function(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
plt.contourf(xx, yy, Z, alpha=0.75)
plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k')
plt.show()
```

6. Controlled Support Vector Machines

Controlled SVM refers to **regularization** using **C parameter**:

- **High C**: Less regularization (tries to classify all points correctly, may overfit)
- **Low C**: More regularization (allows misclassification, prevents overfitting)

```
model = SVC(kernel='linear', C=0.1)  # Lower C -> more regularization
model.fit(X_train, y_train)
```

7. Support Vector Regression (SVR)

SVM is not limited to classification; it can also perform regression tasks, known as **Support Vector Regression (SVR)**.

7.1 How SVR Works?

- Instead of finding a decision boundary, SVR finds a **best-fit hyperplane** that predicts continuous values.
- It uses an **epsilon-insensitive loss function**, meaning errors within a certain margin are ignored.

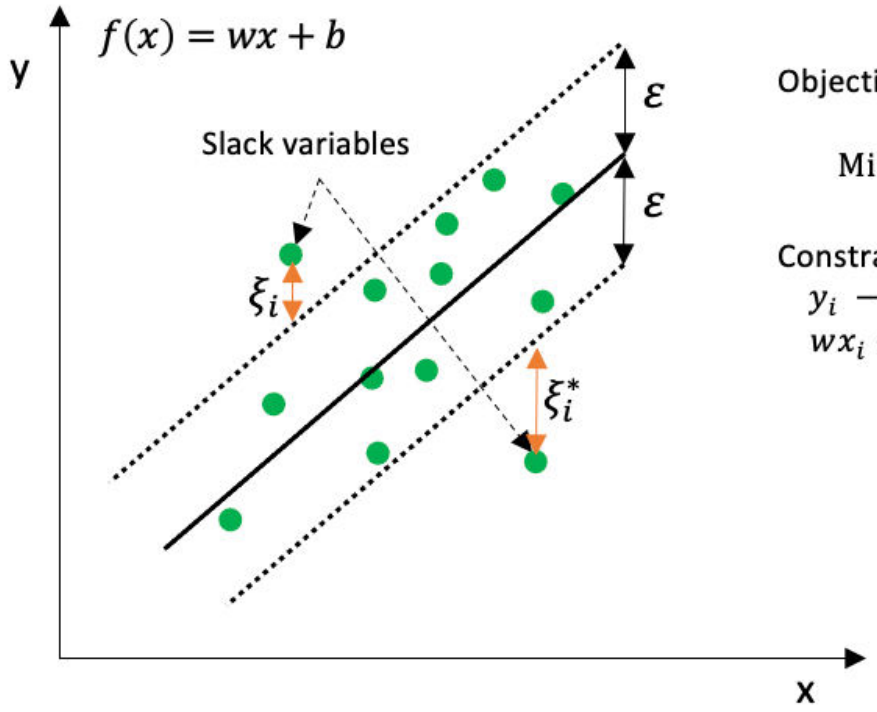
7.2 Scikit-Learn Implementation of SVR

```
from sklearn.svm import SVR

# Generate synthetic data
X = np.sort(5 * np.random.rand(40, 1), axis=0)
y = np.sin(X).ravel()

# Train SVR model
svr_rbf = SVR(kernel='rbf', C=100, gamma=0.1, epsilon=0.1)
svr_rbf.fit(X, y)

# Predictions
y_pred = svr_rbf.predict(X)
plt.scatter(X, y, label='True Data')
plt.plot(X, y_pred, color='red', label='SVR Prediction')
plt.legend()
plt.show()
```



Objective:

$$\text{Minimize: } \frac{1}{2}||w||^2 + C\sum_{i=1}^{l}(\xi_i + \xi_i^*)$$

Constraints:

$$y_i - wx_i - b \leq \varepsilon + \xi_i$$
$$wx_i + b - y_i \leq \varepsilon + \xi_i^*$$
$$\xi_i^*, \xi_i^* \geq 0$$

Univariate linear SVR (allowing for errors)

Conclusion

- **Linear SVM** is ideal for **linearly separable data**.
- **Kernel SVM** transforms data into higher dimensions for non-linear cases.
- **Regularization (C parameter)** controls model complexity.
- **SVR extends SVM for regression tasks**.

1. Basic SVM Questions

Q1. What is Support Vector Machine (SVM)?

A: SVM is a supervised learning algorithm used for classification and regression. It works by finding the optimal hyperplane that maximizes the margin between different classes.

Q2. What are Support Vectors?

A: Support vectors are the data points that lie closest to the decision boundary (hyperplane) and influence its position.

Q3. What is a Hyperplane in SVM?

A: A hyperplane is a decision boundary that separates different classes in SVM. In 2D, it's a line, and in 3D, it's a plane.

Q4. What is the main objective of SVM?

A: The goal of SVM is to maximize the **margin** between the closest data points of different classes, ensuring better generalization.

Q5. What are the advantages of SVM?

A:

- Effective in high-dimensional spaces.
- Works well with small datasets.
- Can handle both linear and non-linear classification using **kernels**.
- Robust to outliers due to the margin maximization principle.

2. SVM Hyperparameters and Optimization

Q6. What is the role of the C parameter in SVM?

A:

- **C (regularization parameter)** controls the trade-off between maximizing the margin and minimizing classification errors.
- **High C**→ Tries to classify all training data correctly (low bias, high variance).
- **Low C**→ Allows some misclassification to generalize better (high bias, low variance).

Q7. What is the Kernel Trick in SVM?

A:

- When data is **not linearly separable**, SVM maps it to a higher dimension where a linear hyperplane can separate it.
- The **kernel trick** helps avoid explicitly computing this transformation, making it computationally efficient.

Q8. What are different types of kernels in SVM?

A:

- **Linear Kernel:** Used for linearly separable data.
- **Polynomial Kernel:** Maps data into a higher-degree polynomial space.
- **Radial Basis Function (RBF) Kernel:** Measures similarity based on distance (commonly used).
- **Sigmoid Kernel:** Mimics neural networks.

Q9. When should you use the RBF kernel over a linear kernel?

A: Use **RBF kernel** when data is **non-linearly separable**, as it maps data into a higher-dimensional space to make it separable.

3. SVM Applications and Comparisons

Q10. What are real-world applications of SVM?

A:

- **Text classification** (spam detection, sentiment analysis).
- **Image classification** (face recognition).
- **Bioinformatics** (cancer detection).
- **Handwriting recognition**.

Q11. How does SVM compare to Logistic Regression?

Feature	SVM	Logistic Regression
Decision Boundary	Maximizes margin	Probability-based
Kernels	Supports kernels	No kernel support
Overfitting	Less prone to overfitting	Can overfit with high dimensions
Computational Cost	Higher with large data	Faster and simpler

Q12. What is the difference between SVM Classification and SVM Regression?

A:

Feature	SVM Classification	SVM Regression
Objective	Separate classes	Predict continuous values
Output	Class labels (0,1)	Real numbers
Hyperplane	Maximizes margin between classes	Fits data within a margin

4. Advanced SVM Questions

Q13. How does SVM handle imbalanced datasets?

A:

- **Class weights:** Adjust the class_weight parameter in Scikit-Learn.
- **Resampling:** Use oversampling (SMOTE) or undersampling techniques.
- **Threshold tuning:** Adjust decision thresholds based on precision-recall.

Q14. Can SVM be used for multi-class classification?

A:

- Yes, using **One-vs-All (OvA)** or **One-vs-One (OvO)** strategies.
- Scikit-Learn's SVC automatically applies one of these techniques.

Q15. How does SVM handle noisy data?

A:

- Noisy data can lead to overfitting.
- **Reduce overfitting** by tuning the C parameter and using soft margins.

5. Coding and Implementation Questions

Q16. How do you implement SVM in Python using Scikit-Learn?

```
from sklearn.svm import SVC
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# Generate synthetic data
X, y = make_classification(n_samples=100, n_features=2, random_state=42)

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train SVM model
model = SVC(kernel='linear', C=1.0)
model.fit(X_train, y_train)

# Predict on test set
y_pred = model.predict(X_test)
```

Q17. How can you tune an SVM model?

A:

- Use GridSearchCV to find optimal values for **C**, **kernel**, and **gamma**.
- Example:

```
from sklearn.model_selection import GridSearchCV

params = {'C': [0.1, 1, 10], 'kernel': ['linear', 'rbf']}
grid_search = GridSearchCV(SVC(), params, cv=5)
grid_search.fit(X_train, y_train)
print(grid_search.best_params_)
```

Q18. How do you evaluate an SVM model?

A: Use classification metrics:

```
from sklearn.metrics import accuracy_score, classification_report

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Practice Links and Blogs for SVM

- **Scikit-Learn SVM Guide**: https://scikit-learn.org/stable/modules/svm.html
- **SVM on Kaggle**: https://www.kaggle.com/general/273769
- **GeeksforGeeks SVM Tutorial**: https://www.geeksforgeeks.org/support-vector-machine/
- **SVM on Machine Learning Mastery**: https://machinelearningmastery.com/support-vector-machines-for-machine-learning/